



PDF

Succinctly

by Ryan Hodson

PDF Succinctly

By
Ryan Hodson

Foreword by Daniel Jebaraj



Copyright © 2012 by Syncfusion Inc.

2501 Aerial Center Parkway

Suite 200

Morrisville, NC 27560

USA

All rights reserved.

I mportant licensing information. Please read.

This book is available for free download from www.syncfusion.com on completion of a registration form.

If you obtained this book from any other source, please register and download a free copy from www.syncfusion.com.

This book is licensed for reading only if obtained from www.syncfusion.com.

This book is licensed strictly for personal, educational use.

Redistribution in any form is prohibited.

The authors and copyright holders provide absolutely no warranty for any information provided.

The authors and copyright holders shall not be liable for any claim, damages, or any other liability arising from, out of, or in connection with the information in this book.

Please do not use this book if the listed terms are unacceptable.

Use shall constitute acceptance of the terms listed.

E dited by

This publication was edited by Stephen Jebaraj, senior product manager, Syncfusion, Inc.

Table of Contents

The Story behind the <i>Succinctly</i> Series of Books	6
Introduction	8
The PDF Standard	8
Chapter 1: Conceptual Overview	9
Header	9
Body	10
Cross-Reference Table	11
Trailer	11
Summary	12
Chapter 2: Building a PDF.....	13
Header	13
Body	13
The Page Tree.....	13
Page(s).....	14
Resources	15
Content	16
Catalog	17
Cross-Reference Table	17
Trailer	17
Compiling the Valid PDF	18
Header Binary.....	18
Content Stream Length.....	19
Cross-Reference Table.....	19
Trailer Dictionary.....	19
Summary	20
Chapter 3: Text Operators.....	21
The Basics	21
Positioning Text	22
Text State Operators.....	23
The Tf Operator	23
The Tc Operator	24
The Tw Operator.....	24
The Tr Operator	25
The Ts Operator	25
The TL Operator	25
Text Positioning Operators.....	26
The Td Operator	27
The T* Operator.....	27
The Tm Operator	27
Text Painting Operators	29
The Tj Operator	29
The ' (Single Quote) Operator	29
The " (Double Quote) Operator	30
The TJ Operator	31
Summary	32
Chapter 4: Graphics Operators.....	33
The Basics	33

Graphics State Operators	34
The w Operator	35
The d Operator	35
The J, j, and M Operators	35
The cm Operator	37
The q and Q Operators	38
The RG, rg, K, and k Operators	38
Path Construction Operators	39
The m Operator	39
The l (lowercase L) Operator	39
The c Operator	40
The h Operator	40
Graphics Painting Operators	41
The S and s Operators	41
The f Operator	41
The B and b Operators	42
The * (asterisk) Operators	42
Summary	43
Chapter 5: Navigation and Annotations	44
Preparations	44
The Document Outline	45
The Initial Destination	48
Hyperlinks	48
Text Annotations	49
Summary	50
Chapter 6: Creating PDFs in C#	51
Disclaimer	51
Installation	51
The Basics	51
Compiling	52
iTextSharp Text Objects	53
Chunks	53
Phrases	54
Paragraphs	54
Lists	55
Formatting a Document	55
Document Dimensions	55
Colors	56
Selecting Fonts	56
Custom Fonts	57
Formatting Text Blocks	58
Summary	59
Conclusion	60

The Story behind the *Succinctly* Series of Books

Daniel Jebaraj, Vice President
Syncfusion, Inc.

Staying on the cutting edge

As many of you may know, Syncfusion is a provider of software components for the Microsoft platform. This puts us in the exciting but challenging position of always being on the cutting edge.

Whenever platforms or tools are shipping out of Microsoft, which seems to be about every other week these days, we have to educate ourselves, quickly.

Information is plentiful but harder to digest

In reality, this translates into a lot of book orders, blog searches, and Twitter scans.

While more information is becoming available on the Internet and more and more books are being published, even on topics that are relatively new, one aspect that continues to inhibit us is the inability to find concise technology overview books.

We are usually faced with two options: read several 500+ page books or scour the Web for relevant blog posts and other articles. Just as everyone else who has a job to do and customers to serve, we find this quite frustrating.

The *Succinctly* series

This frustration translated into a deep desire to produce a series of concise technical books that would be targeted at developers working on the Microsoft platform.

We firmly believe, given the background knowledge such developers have, that most topics can be translated into books that are between 50 and 100 pages.

This is exactly what we resolved to accomplish with the *Succinctly* series. Isn't everything wonderful born out of a deep desire to change things for the better?

The best authors, the best content

Each author was carefully chosen from a pool of talented experts who shared our vision. The book you now hold in your hands, and the others available in this series, are a result of the authors' tireless work. You will find original content that is guaranteed to get you up and running in about the time it takes to drink a few cups of coffee.

Free forever

Syncfusion will be working to produce books on several topics. The books will always be free. Any updates we publish will also be free.

Free? What is the catch?

There is no catch here. Syncfusion has a vested interest in this effort.

As a component vendor, our unique claim has always been that we offer deeper and broader frameworks than anyone else on the market. Developer education greatly helps us market and sell against competing vendors who promise to “enable AJAX support with one click,” or “turn the moon to cheese!”

Let us know what you think

If you have any topics of interest, thoughts, or feedback, please feel free to send them to us at succinctly-series@syncfusion.com.

We sincerely hope you enjoy reading this book and that it helps you better understand the topic of study. Thank you for reading.

Introduction

Adobe Systems Incorporated's Portable Document Format (PDF) is the de facto standard for the accurate, reliable, and platform-independent representation of a paged document. It's the only universally accepted file format that allows pixel-perfect layouts. In addition, PDF supports user interaction and collaborative workflows that are not possible with printed documents.

PDF documents have been in widespread use for years, and dozens of free and commercial PDF readers, editors, and libraries are readily available. However, despite this popularity, it's still difficult to find a succinct guide to the native PDF format. Understanding the internal workings of a PDF makes it possible to dynamically generate PDF documents. For example, a web server can extract information from a database, use it to customize an invoice, and serve it to the customer on the fly.

This book introduces the fundamental components of the native PDF language. With the help of a utility program called [pdftk](#) from PDF Labs, we'll build a PDF document from scratch, learning how to position elements, select fonts, draw vector graphics, and create interactive tables of contents along the way. The goal is to provide enough information to let you start building your own documents without bogging you down with the many complexities of the PDF file format.

In addition, the last chapter of this book provides an overview of the iTextSharp library (<http://itextpdf.com/>). iTextSharp is a C# library that provides an object-oriented wrapper for native PDF elements. Having a C# representation of a document makes it much easier to leverage existing .NET components and streamline the creation of dynamic PDF files.

The sample files created in this book can be downloaded here:
<https://bitbucket.org/syncfusion/pdf-succinctly/>.

The PDF Standard

The PDF format is an open standard maintained by the International Organization for Standardization. The official specification is defined in [ISO 32000-1:2008](#), but Adobe also provides a free, comprehensive guide called [PDF Reference, Sixth Edition, version 1.7](#).

Chapter 1 Conceptual Overview

We'll begin with a conceptual overview of a simple PDF document. This chapter is designed to be a brief orientation before diving in and creating a real document from scratch.

A PDF file can be divided into four parts: a header, body, cross-reference table, and trailer. The header marks the file as a PDF, the body defines the visible document, the cross-reference table lists the location of everything in the file, and the trailer provides instructions for how to start reading the file.

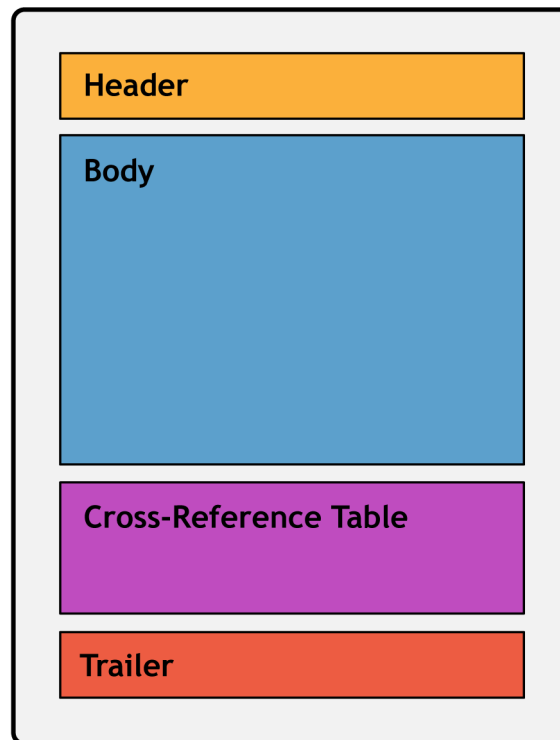


Figure 1: Components of a PDF document

Every PDF file *must* have these four components.

Header

The header is simply a PDF version number and an arbitrary sequence of binary data. The binary data prevents naïve applications from processing the PDF as a text file. This would result in a corrupted file, since a PDF typically consists of both plain text and binary data (e.g., a binary font file can be directly embedded in a PDF).

Body

The body of a PDF contains the entire visible document. The minimum elements required in a valid PDF body are:

- A page tree
- Pages
- Resources
- Content
- The catalog

The **page tree** serves as the root of the document. In the simplest case, it is just a list of the pages in the document. Each **page** is defined as an independent entity with metadata (e.g., page dimensions) and a reference to its resources and content, which are defined separately. Together, the page tree and page objects create the “paper” that composes the document.

Resources are objects that are required to render a page. For example, a single font is typically used across several pages, so storing the font information in an external resource is much more efficient. A **content** object defines the text and graphics that actually show up on the page. Together, content objects and resources define the appearance of an individual page.

Finally, the document’s **catalog** tells applications where to start reading the document. Often, this is just a pointer to the root page tree.

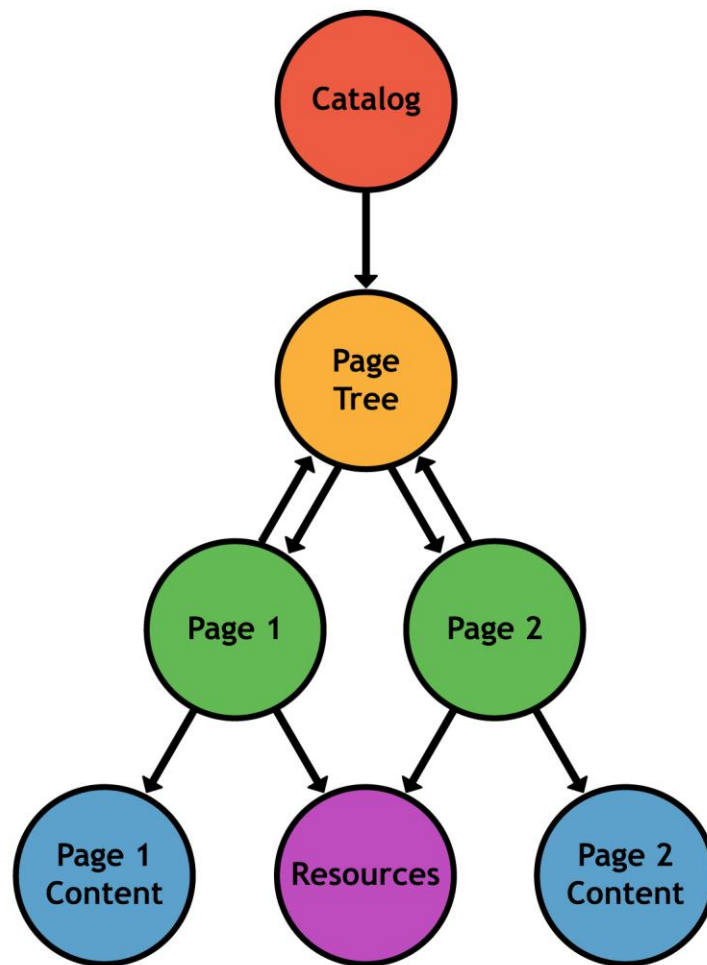


Figure 2: Structure of a document's body

Cross-Reference Table

After the header and the body comes the cross-reference table. It records the byte location of each object in the body of the file. This enables random-access of the document, so when rendering a page, only the objects required for that page are read from the file. This makes PDFs much faster than their PostScript predecessors, which had to read in the entire file before processing it.

Trailer

Finally, we come to the last component of a PDF document. The trailer tells applications how to start reading the file. At minimum, it contains three things:

1. A reference to the catalog which links to the root of the document.
2. The location of the cross-reference table.
3. The size of the cross-reference table.

Since a trailer is all you need to begin processing a document, PDFs are typically read back-to-front: first, the end of the file is found, and then you read backwards until you arrive at the beginning of the trailer. After that, you should have all the information you need to load any page in the PDF.

Summary

To conclude our overview, a PDF document has a header, a body, a cross-reference table, and a trailer. The trailer serves as the entryway to the entire document, giving you access to any object via the cross-reference table, and pointing you toward the root of the document. The relationship between these elements is shown in the following figure.

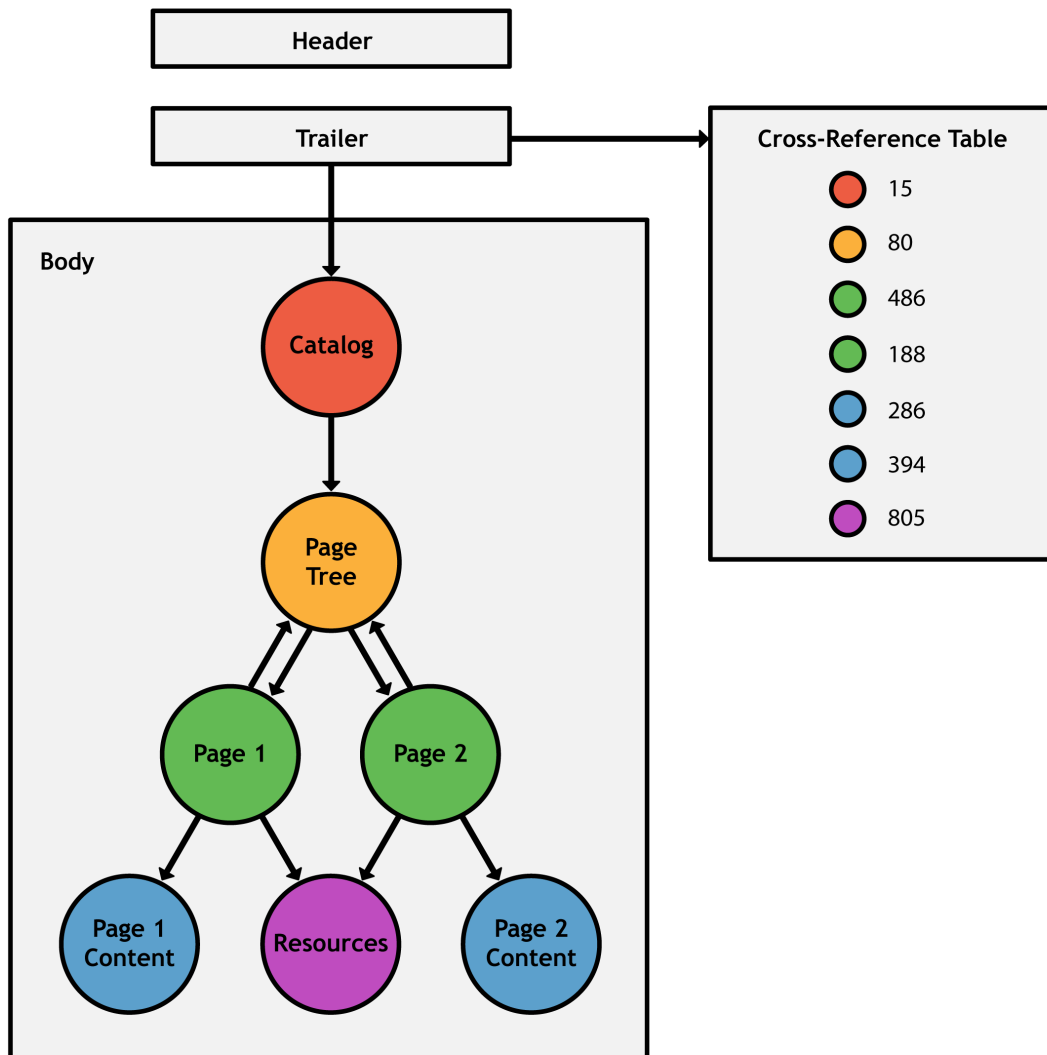


Figure 3: Structure of a PDF document

Chapter 2 Building a PDF

PDFs contain a mix of text and binary, but it's still possible to create them from scratch using nothing but a text editor and a program called `pdftk`. You create the header, body, and trailer on your own, and then the `pdftk` utility goes in and fills in the binary blanks for you. It also manages object references and byte calculations, which is not something you would want to do manually.

First, download `pdftk` from [PDF Labs](http://www.pdf-labs.com/). For Windows users, installation is as simple as unzipping the file and adding the resulting folder to your **PATH**. Running `pdftk --help` from a command prompt should display the help page if installation was successful.

Next, we'll manually create a PDF file for use with `pdftk`. Create a plain text file called **hello-src.pdf** (this file is available at <https://bitbucket.org/syncfusion/pdf-succinctly>) and open it in your favorite text editor.

Header

We'll start by adding a header to **hello-src.pdf**. Remember that the header contains both the PDF version number and a bit of binary data. We'll just add the PDF version and leave the binary data to `pdftk`. Add the following to **hello-src.pdf**.

```
%PDF-1.0
```

The `%` character begins a PDF comment, so the header is really just a special kind of comment.

Body

The body (and hence the entire visible document) is built up using **objects**. Objects are the basic unit of PDF files, and they roughly correspond to the data structures of popular programming languages. For example, PDF has Boolean, numeric, string, array, and dictionary objects, along with streams and names, which are specific to PDF. We'll take a look at each type as the need arises.

The Page Tree

The page tree is a dictionary object containing a list of the pages that make up the document. A minimal page tree contains just one page.

```
1 0 obj
<< /Type /Pages
    /Count 1
    /Kids [2 0 R]
>>
endobj
```

Objects are enclosed in the **obj** and **endobj** tags, and they begin with a unique identification number (1 0). The first number is the *object number*, and the second is the *generation number*. The latter is only used for incremental updates, so all the generation numbers in our examples will be 0. As we'll see in a moment, PDFs use these identifiers to refer to individual objects from elsewhere in the document.

Dictionaries are set off with angle brackets (<< and >>), and they contain key/value pairs. White space is used to separate both the keys from the values *and* the items from each other, which can be confusing. It helps to keep pairs on separate lines, as in the previous example.

The **/Type**, **/Pages**, **/Count**, and **/Kids** keys are called **names**. They are a special kind of data type similar to the constants of high-level programming languages. PDFs often use names as dictionary keys. Names are case-sensitive.

2 0 R is a reference to the object with an identification number of 2 0 (it hasn't been created yet). The **/Kids** key wraps this reference in square brackets, turning it into an array: **[2 0 R]**. PDF arrays can mix and match types, so they are actually more like C#'s **List<object>** than native arrays.

Like dictionaries, PDF arrays are also separated by white space. Again, this can be confusing, since the object reference is also separated by white space. For example, adding a second reference to **/Kids** would look like: **[2 0 R 3 0 R]** (don't actually add this to **hello-src.pdf**, though).

Page(s)

Next, we'll create the second object, which is the only page referenced by **/Kids** in the previous section.

```

2 0 obj
<< /Type /Page
    /MediaBox [0 0 612 792]
    /Resources 3 0 R
    /Parent 1 0 R
    /Contents [4 0 R]
>>
endobj

```

The **/Type** entry always specifies the type of the object. Many times, this can be omitted if the object type can be inferred by context. Note that PDF uses a name to identify the object type—not a literal string.

The **/MediaBox** entry defines the dimensions of the page in points. There are 72 points in an inch, so we’ve just created a standard 8.5 × 11 inch page. **/Resources** points to the object containing necessary resources for the page. **/Parent** points back to the page tree object. Two-way references are quite common in PDF files, since they make it very easy to resolve dependencies in either direction. Finally, **/Contents** points to the object that defines the appearance of the page.

Resources

The third object is a resource defining a font configuration.

```

3 0 obj
<< /Font
    << /F0
        << /Type /Font
            /BaseFont /Times-Roman
            /Subtype /Type1
        >>
    >>
>>
endobj

```

The **/Font** key contains a whole dictionary, opposed to the name/value pairs we’ve seen previously (e.g., **/Type /Page**). The font we configured is called **/F0**, and the font face we selected is **/Times-Roman**. The **/Subtype** is the format of the font file, and **/Type1** refers to the PostScript type 1 file format.

The specification defines 14 “standard” fonts that all PDF applications should support.